

```

while (in >> s) {
    auto trans = findBook(files, s); // 销售了这本书的书店
    if (trans.empty()) {
        cout << s << " not found in any stores" << endl;
        continue; // 获得下一本要查找的书
    }
    for (const auto &store : trans) // 对每家销售了给定书籍的书店
        // get<n>返回 store 中 tuple 的指定的成员
        os << "store " << get<0>(store) << " sales: "
            << accumulate(get<1>(store), get<2>(store),
                          Sales_data(s))
            << endl;
    }
}

```

while 循环反复读取名为 in 的 istream 来获得下一本要处理的书。我们调用 findBook 来检查 s 是否存在，并将结果赋予 trans。我们使用 auto 来简化 trans 类型的代码编写，它是一个 tuple 的 vector。

如果 trans 为空，表示没有关于 s 的销售记录。在此情况下，我们打印一条信息并返回，执行下一步 while 循环来获取下一本要查找的书。

for 循环将 store 绑定到 trans 中的每个元素。由于不希望改变 trans 中的元素，我们将 store 声明为 const 的引用。我们使用 get 来打印相关数据：get<0>表示对应书店的索引、get<1>表示第一条交易记录的迭代器、get<2>表示尾后位置的迭代器。

由于 Sales_data 定义了加法运算符（参见 14.3 节，第 497 页），因此我们可以用标准库的 accumulate 算法（参见 10.2.1 节，第 338 页）来累加销售记录。我们用 Sales_data 的接受一个 string 参数的构造函数（参见 7.1.4 节，第 236 页）来初始化一个 Sales_data 对象，将此对象传递给 accumulate 作为求和的起点。此构造函数用给定的 string 初始化 bookNo，并将 units_sold 和 revenue 成员置为 0。

723

17.1.2 节练习

练习 17.4：编写并测试你自己版本的 findBook 函数。

练习 17.5：重写 findBook，令其返回一个 pair，包含一个索引和一个迭代器 pair。

练习 17.6：重写 findBook，不使用 tuple 或 pair。

练习 17.7：解释你更倾向于哪个版本的 findBook，为什么。

练习 17.8：在本节最后一段代码中，如果我们将 Sales_data() 作为第三个参数传递给 accumulate，会发生什么？

17.2 bitset 类型

在 4.8 节（第 135 页）中我们介绍了将整型运算对象当作二进制位集合处理的一些内置运算符。标准库还定义了 **bitset** 类，使得位运算的使用更为容易，并且能够处理超过最长整型类型大小的位集合。bitset 类定义在头文件 `bitset` 中。

17.2.1 定义和初始化 bitset

表 17.2 列出了 bitset 的构造函数。bitset 类是一个类模板，它类似 array 类，具有固定的大小（参见 9.2.4 节，第 301 页）。当我们定义一个 bitset 时，需要声明它包含多少个二进制位：

```
bitset<32> bitvec(1U); // 32 位；低位为 1，其他位为 0
```

大小必须是一个常量表达式（参见 2.4.4 节，第 58 页）。这条语句定义 bitvec 为一个包含 32 位的 bitset。就像 vector 包含未命名的元素一样，bitset 中的二进制位也是未命名的，我们通过位置来访问它们。二进制位的位置是从 0 开始编号的。因此，bitvec 包含编号从 0 到 31 的 32 个二进制位。编号从 0 开始的二进制位被称为低位（low-order），编号到 31 结束的二进制位被称为高位（high-order）。

表 17.2: 初始化 bitset 的方法	
bitset<n> b;	b 有 n 位；每一位均为 0。此构造函数是一个 constexpr（参见 7.5.6 节，第 267 页）
bitset<n> b(u);	b 是 unsigned long long 值 u 的低 n 位的拷贝。如果 n 大于 unsigned long long 的大小，则 b 中超出 unsigned long long 的高位被置为 0。此构造函数是一个 constexpr（参见 7.5.6 节，第 267 页）
bitset<n> b(s, pos, m, zero, one);	b 是 string s 从位置 pos 开始 m 个字符的拷贝。s 只能包含字符 zero 或 one；如果 s 包含任何其他字符，构造函数会抛出 invalid_argument 异常。字符在 b 中分别保存为 zero 和 one。pos 默认为 0，m 默认为 string::npos，zero 默认为 '0'，one 默认为 '1'
bitset<n> b(cp, pos, m, zero, one);	与上一个构造函数相同，但从 cp 指向的字符数组中拷贝字符。如果未提供 m，则 cp 必须指向一个 C 风格字符串。如果提供了 m，则从 cp 开始必须至少有 m 个 zero 或 one 字符
接受一个 string 或一个字符指针的构造函数是 explicit 的（参见 7.5.4 节，第 265 页）。在新标准中增加了为 0 和 1 指定其他字符的功能。	

用 unsigned 值初始化 bitset

当我们使用一个整型值来初始化 bitset 时，此值将被转换为 unsigned long long 类型并被当作位模式来处理。bitset 中的二进制位将是此模式的一个副本。如果 bitset 的大小大于一个 unsigned long long 中的二进制位数，则剩余的高位被置为 0。如果 bitset 的大小小于一个 unsigned long long 中的二进制位数，则只使用给定值中的低位，超出 bitset 大小的高位被丢弃：

```
// bitvec1 比初始值小；初始值中的高位被丢弃
bitset<13> bitvec1(0xbeef); // 二进制位序列为 1111011101111
// bitvec2 比初始值大；它的高位被置为 0
bitset<20> bitvec2(0xbeef); // 二进制位序列为 00001011111011101111
// 在 64 位机器中，long long OULL 是 64 个 0 比特，因此 ~OULL 是 64 个 1
bitset<128> bitvec3(~OULL); // 0~63 位为 1；63~127 位为 0
```

从一个 string 初始化 bitset

我们可以从一个 string 或一个字符数组指针来初始化 bitset。两种情况下，字符都直接表示位模式。与往常一样，当我们使用字符串表示数时，字符串中下标最小的字符对应高位，反之亦然：

```
bitset<32> bitvec4("1100"); // 2、3 两位为 1，剩余两位为 0
```

如果 string 包含的字符数比 bitset 少，则 bitset 的高位被置为 0。

Note

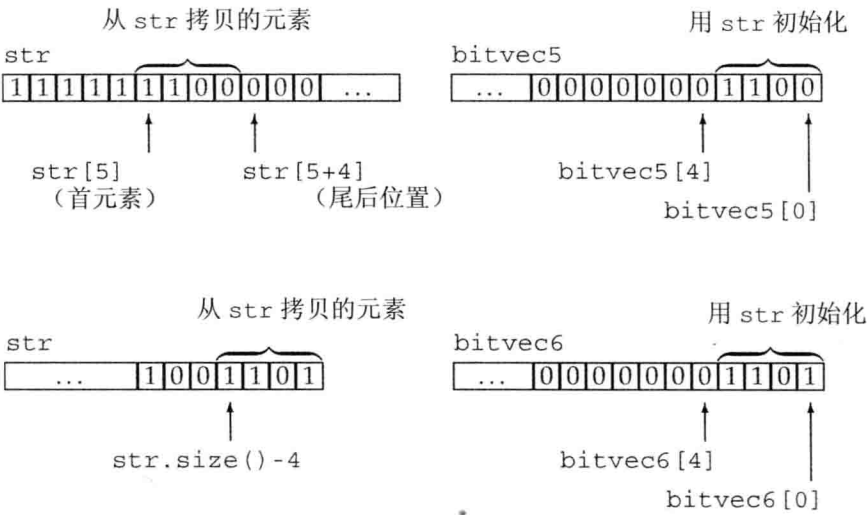
string 的下标编号习惯与 bitset 恰好相反:string 中下标最大的字符(最右字符)用来初始化 bitset 中的低位(下标为 0 的二进制位)。当你用一个 string 初始化一个 bitset 时，要记住这个差别。

725

我们不必使用整个 string 来作为 bitset 的初始值，可以只用一个子串作为初始值：

```
string str("1111111000000011001101");
bitset<32> bitvec5(str, 5, 4); // 从 str[5] 开始的四个二进制位，1100
bitset<32> bitvec6(str, str.size()-4); // 使用最后四个字符
```

此处，bitvec5 用 str 中从 str[5] 开始的长度为 4 的子串进行初始化。与往常一样，子串的最右字符表示最低位。因此，bitvec5 中第 3 位到第 0 位被设置为 1100，剩余位被设置为 0。传递给 bitvec6 的初始值是一个 string 和一个开始位置，因此 bitvec6 用 str 中倒数第四个字符开始的子串进行初始化。bitvec6 中剩余二进制位被初始化为 0。下图说明了这两个初始化过程



17.2.1 节练习

练习 17.9: 解释下列每个 bitset 对象所包含的位模式:
(a) `bitset<64> bitvec(32);`
(b) `bitset<32> bv(1010101);`
(c) `string bstr; cin >> bstr; bitset<8>bv(bstr);`

17.2.2 bitset 操作

bitset 操作（参见表 17.3）定义了多种检测或设置一个或多个二进制位的方法。bitset 类还支持我们在 4.8 节（第 136 页）中介绍过的位运算符。这些运算符用于 bitset 对象的含义与内置运算符用于 unsigned 运算对象相同。

表 17.3: bitset 操作	
b.any()	b 中是否存在置位的二进制位
b.all()	b 中所有位都置位了吗
b.none()	b 中不存在置位的二进制位吗
b.count()	b 中置位的位数
b.size()	一个 constexpr 函数（参见 2.4.4 节，第 58 页），返回 b 中的位数
b.test(pos)	若 pos 位置的位是置位的，则返回 true，否则返回 false
b.set(pos,v)	将位置 pos 处的位设置为 bool 值 v。v 默认为 true。如果未传递实参，则将 b 中所有位置位
b.set()	
b.reset(pos)	将位置 pos 处的位复位或将 b 中所有位复位
b.reset()	
b.flip(pos)	改变位置 pos 处的位的状态或改变 b 中每一位的状态
b.flip()	
b[pos]	访问 b 中位置 pos 处的位；如果 b 是 const 的，则当该位置位时 b[pos] 返回一个 bool 值 true，否则返回 false
b.to_ulong()	返回一个 unsigned long 或一个 unsigned long long 值，其位模式与 b 相同。如果 b 中位模式不能放入指定的结果类型，则抛出一个 overflow_error 异常
b.to_ullong()	
b.to_string(zero, one)	返回一个 string，表示 b 中的位模式。zero 和 one 的默认值分别为 0 和 1，用来表示 b 中的 0 和 1
os << b	将 b 中二进制位打印为字符 1 或 0，打印到流 os
is >> b	从 is 读取字符存入 b。当下一个字符不是 1 或 0 时，或是已经读入 b.size() 个位时，读取过程停止

726

count、size、all、any 和 none 等几个操作都不接受参数，返回整个 bitset 的状态。其他操作——set、reset 和 flip 则改变 bitset 的状态。改变 bitset 状态的成员函数都是重载的。对每个函数，不接受参数的版本对整个集合执行给定的操作；接受一个位置参数的版本则对指定位执行操作：

```
bitset<32> bitvec(1U);           // 32 位；低位为 1，剩余位为 0
bool is_set = bitvec.any();      // true，因为有 1 位置位
bool is_not_set = bitvec.none(); // false，因为有 1 位置位了
bool all_set = bitvec.all();     // false，因为只有 1 位置位
size_t onBits = bitvec.count();  // 返回 1
size_t sz = bitvec.size();       // 返回 32
bitvec.flip(); // 翻转 bitvec 中的所有位
bitvec.reset(); // 将所有位复位
bitvec.set(); // 将所有位置位
```

当 bitset 对象的一个或多个位置位（即，等于 1）时，操作 any 返回 true。相反，当所有位复位时，none 返回 true。新标准引入了 all 操作，当所有位置位时返回 true。

727

操作 `count` 和 `size` 返回 `size_t` 类型的值 (参见 3.5.2 节, 第 103 页), 分别表示对象中置位的位数或总位数。函数 `size` 是一个 `constexpr` 函数, 因此可以用在要求常量表达式的地方 (参见 2.4.4 节, 第 58 页)。

成员 `flip`、`set`、`reset` 及 `test` 允许我们读写指定位置的位:

```
bitvec.flip(0);           // 翻转第一位
bitvec.set(bitvec.size() - 1); // 置位最后一位
bitvec.set(0, 0);         // 复位第一位
bitvec.reset(i);          // 复位第 i 位
bitvec.test(0);           // 返回 false, 因为第一位是复位的
```

下标运算符对 `const` 属性进行了重载。`const` 版本的下标运算符在指定位置位时返回 `true`, 否则返回 `false`。非 `const` 版本返回 `bitset` 定义的一个特殊类型, 它允许我们操纵指定位的值:

```
bitvec[0] = 0;             // 将第一位复位
bitvec[31] = bitvec[0];    // 将最后一位设置为与第一位一样
bitvec[0].flip();          // 翻转第一位
~bitvec[0];                // 等价操作, 也是翻转第一位
bool b = bitvec[0];        // 将 bitvec[0] 的值转换为 bool 类型
```

提取 `bitset` 的值

`to_ulong` 和 `to_ullong` 操作都返回一个值, 保存了与 `bitset` 对象相同的位模式。只有当 `bitset` 的大小小于等于对应的大小 (`to_ulong` 为 `unsigned long`, `to_ullong` 为 `unsigned long long`) 时, 我们才能使用这两个操作:

```
unsigned long ulong = bitvec3.to_ulong();
cout << "ulong = " << ulong << endl;
```



如果 `bitset` 中的值不能放入给定类型中, 则这两个操作会抛出一个 `overflow_error` 异常 (参见 5.6 节, 第 173 页)。

`bitset` 的 IO 运算符

输入运算符从一个输入流读取字符, 保存到一个临时的 `string` 对象中。直到读取的字符数达到对应 `bitset` 的大小时, 或是遇到不是 1 或 0 的字符时, 或是遇到文件尾或输入错误时, 读取过程才停止。随即用临时 `string` 对象来初始化 `bitset` (参见 17.2.1 节, 第 642 页)。如果读取的字符数小于 `bitset` 的大小, 则与往常一样, 高位将被置为 0。

输出运算符打印一个 `bitset` 对象中的位模式:

```
bitset<16> bits;
cin >> bits; // 从 cin 读取最多 16 个 0 或 1
cout << "bits: " << bits << endl; // 打印刚刚读取的内容
```

728

使用 `bitset`

为了说明如何使用 `bitset`, 我们重新实现 4.8 节 (第 137 页) 中的评分程序, 用 `bitset` 代替 `unsigned long` 表示 30 个学生的测验结果——“通过/失败”:

```
bool status;
// 使用位运算符的版本
unsigned long quizA = 0;           // 此值被当作位集合使用
```

```
quizA |= 1UL << 27;           // 指出第 27 个学生通过了测验
status = quizA & (1UL << 27); // 检查第 27 个学生是否通过了测验
quizA &= ~(1UL << 27);       // 第 27 个学生未通过测验
// 使用标准库类 bitset 完成等价的工作
bitset<30> quizB;             // 每个学生分配一位, 所有位都被初始化为 0
quizB.set(27);               // 指出第 27 个学生通过了测验
status = quizB[27];          // 检查第 27 个学生是否通过了测验
quizB.reset(27);             // 第 27 个学生未通过测验
```

17.2.2 节练习

练习 17.10: 使用序列 1、2、3、5、8、13、21 初始化一个 bitset，将这些位置置位。对另一个 bitset 进行默认初始化，并编写一小段程序将其恰当的位置位。

练习 17.11: 定义一个数据结构，包含一个整型对象，记录一个包含 10 个问题的真/假测验的解答。如果测验包含 100 道题，你需要对数据结构做出什么改变（如果需要的话）？

练习 17.12: 使用前一题中的数据结构，编写一个函数，它接受一个问题编号和一个表示真/假解答的值，函数根据这两个参数更新测验的解答。

练习 17.13: 编写一个整型对象，包含真/假测验的正确答案。使用它来为前两题中的数据结构生成测验成绩。

17.3 正则表达式

正则表达式（regular expression）是一种描述字符序列的方法，是一种极其强大的计算工具。但是，用于定义正则表达式的描述语言已经大大超出了本书的范围。因此，我们重点介绍如何使用 C++正则表达式库（RE 库），它是新标准库的一部分。RE 库定义在头文件 `regex` 中，它包含多个组件，列于表 17.4 中。

C++
11

表 17.4: 正则表达式库组件	
<code>regex</code>	表示有一个正则表达式的类
<code>regex_match</code>	将一个字符序列与一个正则表达式匹配
<code>regex_search</code>	寻找第一个与正则表达式匹配的子序列
<code>regex_replace</code>	使用给定格式替换一个正则表达式
<code>sregex_iterator</code>	迭代器适配器，调用 <code>regex_search</code> 来遍历一个 <code>string</code> 中所有匹配的子串
<code>smatch</code>	容器类，保存在 <code>string</code> 中搜索的结果
<code>ssub_match</code>	<code>string</code> 中匹配的子表达式的结果



如果你还不熟悉正则表达式的使用，你应该浏览这一节，以获得正则表达式可以做什么的一些概念。

`regex` 类表示一个正则表达式。除了初始化和赋值之外，`regex` 还支持其他一些操作。表 17.6（第 647 页）列出了 `regex` 支持的操作。

函数 `regex_match` 和 `regex_search` 确定一个给定字符序列与一个给定 `regex`